LAB 9 SUPPLEMENTARY REPORT

Porting the Hybrid Mandelbrot Renderer to an AMD ROCm Cluster

Companion to the Lab 9 Hybrid MPI + OpenMP report

22 October 2026

This supplement contains the AMD ROCm port, the PyTorch GPU code, the measured comparison and the limitations of that comparison.

Abstract

The main Lab 9 report measured a hybrid MPI + OpenMP Mandelbrot renderer on a four-vCPU Ubuntu VM. This supplement adds a second execution path: the same image is rendered by a PyTorch ROCm port on an AMD Radeon 8060S GPU in a Jupyter container. The original VM results were serial 0.725 s, static 2x2 0.326 s and dynamic 2x2 0.1833 s. On the AMD container the C serial program took 0.2570 s and the PyTorch ROCm GPU port took 0.18985 s (mean of three runs). The GPU is therefore 1.35x faster than the C serial program in the same container. A pixel comparison also found 345,928 of 480,000 pixels different from the AMD C serial image; the analysis traces most of that to a palette portability issue in the C code and to endpoint sampling in the GPU grid, not to a GPU arithmetic error. The AMD container did not have mpicc or mpirun, so its static and dynamic 2x2 runs were skipped; the comparison with the original MPI + OpenMP result is therefore cross-environment and is reported only as a reference. The supplement explains the image difference, why the GPU gain is modest, why the missing MPI rows matter, and what must be fixed or installed for a controlled comparison.

1 Purpose and scope

The main report answers the Lab 9 question for a four-vCPU VM: MPI divides the image into row bands, OpenMP divides each band between threads, and dynamic scheduling removes most of the load imbalance. This supplement does not replace that report. It records a separate experiment on an AMD ROCm cluster and compares the two execution paths where a comparison is meaningful.

The supplement has three goals:

- document the AMD ROCm environment and the PyTorch GPU implementation;
- report the measured C serial and GPU times and compare them with the original serial, static and dynamic results;
- state clearly which comparison is controlled and which is only a cross-environment reference.

The complete GPU code is listed in Appendix A. The commands needed to reproduce the run are in Appendix B. The main report remains the authoritative source for the MPI + OpenMP decomposition, the C source and the single-node measurements.

2 AMD ROCm environment

Table 1. AMD test environment.

Item	Configuration
Platform	JupyterLab container, AMD ROCm cluster
Python	3.12.3
PyTorch	2.9.1+rocm7.13.0
HIP / ROCm	7.13.99004-3309c6114a
GPU	Radeon 8060S Graphics (integrated RDNA class GPU)
CPU compiler	gcc available
MPI	mpicc / mpirun not installed at run time
Image	800 x 600, max_iter = 1000, one sample per pixel

The container is a single Jupyter node. The MPI experiment in the main report used four vCPUs; the AMD experiment uses one container CPU process for the C serial reference and one AMD GPU for the PyTorch port. These are different hardware and software environments, which is why the comparison is split into a same-environment part and a cross-environment part.

3 Port design and implementation

3.1 Two execution paths

The notebook has two independent paths. The first compiles the original C sources with gcc. If mpicc and mpirun are available it also compiles the static and dynamic MPI + OpenMP variants; if they are missing the MPI part is skipped and the GPU path still runs. The second path reimplements the same escape-time algorithm with PyTorch tensors and runs it on the AMD GPU through ROCm.

The C path keeps the original memory layout: one byte per colour channel, one PPM file, and an FNV-1a hash over the RGB bytes. The GPU path writes the same PPM format and computes the same two checks, so the images and hashes can be compared directly.

3.2 GPU algorithm

The GPU kernel is expressed as tensor operations rather than a hand-written HIP kernel. The complex plane is sampled with two one-dimensional tensors, broadcast to an 800 x 600 grid. Four tensors hold zr, zi, the iteration count and an alive mask. Each iteration performs:

- zr2 = zr*zr and zi2 = zi*zi;
- alive = alive AND (zr2 + zi2 <= 4);
- next_zr = zr2 - zi2 + cr and next_zi = 2*zr*zi + ci;
- zr and zi are updated only where alive is true;
- counts is incremented only where alive is true.

The loop runs for max_iter iterations. Pixels that never escape keep counts = max_iter and are coloured black. Escaping pixels use the same square-root palette as the C version. The default dtype is float64, to stay close to the C double; if the device cannot run float64 the notebook falls back to float32.

The vectorised form is short and easy to verify, but it is not a fused GPU kernel. Every iteration creates temporary tensors and launches several elementwise kernels. That implementation choice matters for the interpretation of the timing and is discussed in Section 5.

3.3 Determinism and comparison method

The renderer has no random number generator. A pixel colour is a pure function of its coordinates and max_iter, so the order of GPU work cannot change the result. The C build uses -ffp-contract=off; the GPU path cannot request the same contraction control, so a boundary pixel can differ. The notebook therefore reports both hashes and counts the number of pixels that differ from the C serial image. In the AMD run that count was large, and Section 4.1 shows that the dominant causes were the C palette and the sampling grid, not the GPU iteration arithmetic.

Speed-up is always defined relative to a serial baseline in the same environment:

```
S = T(serial) / T(parallel)      E = S / W
```

For the original VM, T(serial) = 0.725 s and W is the number of workers. For the AMD container, the only controlled baseline is the measured C serial time, because the AMD MPI + OpenMP configurations

were not run. A ratio against the original VM serial is reported only as a cross-environment reference and is labelled as such.

4 Results

Table 2 collects the original VM results and the AMD results. The original static and dynamic values are from the main report; the AMD C serial value is a single run in this container, and the GPU value is the mean of three runs after a warm-up. The AMD static and dynamic 2x2 rows are absent because the container did not have an MPI toolchain.

Table 2. Original VM and AMD cluster measurements (800 x 600, max_iter = 1000).

Environment	Configuration	Time (s)	Speed-up vs own serial	Relative to original VM serial
Original 4-vCPU VM	serial 1x1	0.725	1.00x	1.00x
Original 4-vCPU VM	static 2x2	0.326	2.22x	2.22x
Original 4-vCPU VM	dynamic 2x2	0.1833	3.95x	3.95x
AMD container	C serial	0.2570	1.00x	2.82x
AMD container	PyTorch ROCm GPU	0.18985	1.35x	3.82x

The last column is not a controlled speed-up: it divides the original VM serial time by a time measured on different hardware. It is included only to answer the natural question "how does this compare with the old numbers?", and it should not be read as a parallel scaling result.

```
WARNING: C static 2x2 and/or dynamic 2x2 were NOT run.
This happens when mpicc or mpirun is missing; the notebook skips MPI.
Install OpenMPI, restart the kernel, then Run All from the first cell.

environment          configuration        time (s)  vs own serial  vs orig. serial
Original 4-vCPU VM   serial 1x1             0.7250          1.00x            1.00x
Original 4-vCPU VM   static 2x2             0.3260          2.22x            2.22x
Original 4-vCPU VM   dynamic 2x2            0.1833          3.95x            3.95x
AMD cluster          C serial               0.2570          1.00x            2.82x
AMD cluster          PyTorch ROCm GPU       0.1898          1.35x            3.82x
```

Figure 1. Notebook output: the warning that MPI rows were skipped, and the comparison table for the original VM and the AMD container.

4.1 The image comparison and the C palette

The first AMD run produced the timings in Table 2, but the pixel comparison against the AMD C serial image was much worse than the "boundary pixel" expectation: 345,928 of 480,000 pixels differed, and the maximum channel difference was 255. The GPU image itself is not random; its outer regions are purple/yellow, while the AMD C serial image is cyan/magenta. The difference has two concrete causes.

First, the C palette does not clamp the lower bound of the green and blue components. The code computes fmin(1.0, 2.0*t - 0.5) and fmin(1.0, 1.0 - 2.0*t); when t is small these expressions are negative. Converting a negative double to unsigned char is undefined in C. On the AMD x86 container the value wraps to a large byte (cyan/magenta), while on the original ARM VM it effectively becomes zero (purple) and the GPU code clamps it to zero (purple). This single portability bug changes most pixels and explains why the AMD C hash is a4d33165df85fdaf while the GPU hash is ea795bf4c1bf5673.

Second, the first GPU version used torch.linspace(xmin, xmax, width) and torch.linspace(ymax, ymin, height). That includes the endpoints and therefore samples pixel edges, whereas the C code samples pixel centres with (px + 0.5) / width. The half-pixel shift moves the fractal boundary and changes the escape count for pixels near it. The corrected formulas are:

```
# C palette: clamp the lower bound before the unsigned-char cast
double rr = fmin(1.0, fmax(0.0, 2.0 * t));
double gg = fmin(1.0, fmax(0.0, 2.0 * t - 0.5));
double bb = fmin(1.0, fmax(0.0, 1.0 - 2.0 * t));
```

```
# GPU grid: sample pixel centres, not the endpoints
x = xmin + (xmax - xmin) * (torch.arange(width,  device=device, dtype=dtype) + 0.5) / width
y = ymax - (ymax - ymin) * (torch.arange(height, device=device, dtype=dtype) + 0.5) / height
```

The corrected notebook in the repository uses the pixel-centre grid; the C palette patch is listed here for a controlled comparison. With both changes the colour conversion becomes deterministic and the remaining difference should be limited to floating-point boundary pixels, not the whole image. The timing result is not invalidated by this correction: both versions perform the same 800 x 600 x 1000 escape-time work, and the measured GPU mean remains a useful estimate of the PyTorch path.

Table 3. Pixel comparison diagnostics from the first AMD run.

Quantity	Result
Pixels differing from AMD C serial	345,928 / 480,000 (72.1%)
Maximum channel difference	255
AMD C serial FNV-1a	a4d33165df85fdaf
GPU FNV-1a	ea795bf4c1bf5673
C outer colour (AMD x86)	cyan/magenta due to negative-to-unsigned-char wrap
GPU outer colour	purple/yellow due to explicit clamp to zero

This is a correctness finding about the port, not a claim that the GPU arithmetic is wrong. The GPU follows the intended clamped palette and the report image; the AMD C binary follows the undefined behaviour of its platform. The lesson is that a byte-for-byte hash comparison across architectures requires the palette and sampling rules to be made explicit first.

4.2 Controlled ratios

The controlled AMD comparison is:

Table 4. Ratios with an explicit interpretation.

Comparison	Ratio	Interpretation
AMD GPU vs AMD C serial	0.2570 / 0.18985 = 1.35x	Controlled, same container: GPU is faster than one C serial process
AMD GPU vs original VM dynamic 2x2	0.18333 / 0.18985 = 0.97x	Cross-environment reference: GPU is about 3.6% slower than the old 4-worker result
AMD GPU vs original VM serial	0.725 / 0.18985 = 3.82x	Cross-environment reference only
AMD C serial vs original VM serial	0.725 / 0.2570 = 2.82x	Hardware/compiler difference, not a parallel speed-up

The key result is the 1.35x GPU gain over the C serial program in the same container. The 0.97x comparison with the original dynamic 2x2 is almost a tie, but it is not evidence that the GPU is slower

than MPI + OpenMP: the two numbers come from different CPUs, different memory systems, different compilers and a different number of workers. The honest statement is that the AMD GPU result is comparable to the old four-worker VM result, while the controlled MPI comparison on the AMD container is still missing.

5 Why the GPU speed-up is modest

A 1.35x gain over one CPU core is much smaller than the 3.95x obtained from four CPU workers. The following effects explain the gap; they are properties of this port and this workload, not evidence that the GPU is slow in general.

5.1 Kernel-launch overhead

The PyTorch loop runs 1000 Python-level iterations. Each iteration launches several elementwise kernels (multiply, add, compare, where, conversion and accumulation). For an 800 x 600 image each kernel is small, so launch and synchronisation overhead is a significant fraction of the total. A fused HIP kernel would remove most of these launches; the notebook deliberately uses a readable tensor formulation instead.

5.2 Temporary tensors and memory traffic

torch.where and the intermediate zr2, zi2, next_zr and next_zi tensors allocate or stream through memory on every iteration. The arithmetic intensity is low: each pixel is read and written several times per iteration for only a few floating-point operations. On an integrated Radeon 8060S the GPU shares memory with the host, so the achievable bandwidth is below a discrete GPU with dedicated HBM.

5.3 Float64 and the integrated GPU

The port uses float64 to stay close to the C double. Consumer RDNA GPUs typically have much lower float64 throughput than float32 throughput, and the 8060S is an integrated part rather than a data-centre accelerator. The float64 choice is good for numerical agreement but not for peak GPU throughput. A float32 or mixed-precision kernel would be faster, at the cost of larger pixel differences from the C reference.

5.4 Small problem and a fast serial baseline

The image is only 1.4 MB and the C serial program takes 0.257 s in this container. Fixed costs such as context setup, the first kernel launch and the Python loop are not amortised over a long run. A larger image or a larger max_iter would give the GPU more work per launch and would change the ratio. The 1.35x result is therefore a statement about this 800 x 600 workload, not a general GPU speed-up claim.

6 The missing MPI rows on the AMD container

The notebook checks for gcc, mpicc and mpirun before compiling. The AMD container had gcc but did not have mpicc or mpirun, so the build cell skipped the static and dynamic 2x2 programs and printed:

```
WARNING: C static 2x2 and/or dynamic 2x2 were NOT run.
This happens when mpicc or mpirun is missing; the notebook skips MPI.
Install OpenMPI, restart the kernel, then Run All from the first cell.
```

The container also blocks privilege escalation: sudo reports that the "no new privileges" flag is set. The apt route is therefore unavailable even though apt exists. The available rootless route is conda, mamba

or micromamba from conda-forge. The notebook searches common locations such as /opt/conda/bin/conda and installs openmpi when it finds a manager.

This missing toolchain has a direct consequence for the report: there is no AMD static 2x2 or dynamic 2x2 measurement. The original VM static and dynamic values in Table 2 are therefore not AMD measurements. Any statement that compares the AMD GPU with "MPI + OpenMP on the same machine" would be unsupported until the MPI rows are actually run.

To complete the controlled comparison, install a rootless OpenMPI and rerun the notebook:

```
python -m pip install --user mpi4py   # optional
conda install -y -c conda-forge \
  openmpi gcc_linux-64 gxx_linux-64


# or, without conda:
curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest \
  | tar -xvj -C ~/.local bin/micromamba
~/.local/bin/micromamba install -y -p ~/.local/mpi \
  -c conda-forge openmpi
export PATH="$HOME/.local/mpi/bin:$PATH"
```

After installation, mpicc and mpirun should be visible, the kernel should be restarted, and the notebook should be run from the first cell. The static and dynamic rows would then appear in Table 2 and the AMD GPU could be compared with a same-container MPI + OpenMP baseline.

7 Comparison analysis

The original VM experiment isolates the scheduling effect: serial 0.725 s, static 2x2 0.326 s, dynamic 2x2 0.1833 s. The static result is limited by the 150-row block containing the expensive middle rows; dynamic 8-row chunks let free threads take those rows and reach 3.95x. That conclusion is unchanged by this supplement.

The AMD experiment answers a different question: how does a PyTorch ROCm port compare with the C code on the hardware that is actually available? The answer is 1.35x over the AMD C serial program. The GPU result is 0.18985 s, which is close to the original VM dynamic result of 0.1833 s, but the two numbers should not be subtracted from one another: the VM used four CPU workers and a different compiler, while the AMD run used one GPU and one CPU serial process.

The comparison can be summarised in three statements:

- Within the AMD container, the GPU port is 1.35x faster than the C serial program; this is the only controlled AMD speed-up in this supplement.
- The original VM MPI + OpenMP result remains 3.95x faster than its own serial baseline; the GPU result does not replace or invalidate that measurement.
- The cross-environment ratios (3.82x against the old serial, 0.97x against the old dynamic 2x2) are references, not controlled results; the missing AMD MPI rows are the main gap.

The 0.97x value should not be reported as "the GPU is slower than MPI + OpenMP". It says that a single integrated GPU running a Python-level PyTorch loop is approximately as fast as four CPU workers running the optimized C hybrid on a different machine. That is a useful observation, but it is not a like-for-like comparison.

8 Limitations and conclusions

The main limitations are:

- The AMD container had no MPI toolchain, so the AMD static and dynamic 2x2 configurations were not measured.
- The AMD C serial time is a single observation; the GPU time is a three-run mean after one warm-up.
- The GPU port is a readable PyTorch tensor implementation, not a fused HIP kernel; its launch and memory overhead is part of the measured 0.18985 s.
- The image is small and the serial baseline is short, so fixed overheads are not amortised.
- The original VM and the AMD cluster are different hardware; only the ratios inside each environment are controlled.

The port implements the same escape-time rule and PPM layout, and its measured gain over the AMD C serial program is 1.35x. It is not yet a byte-identical port: the first run exposed the C palette portability issue and the endpoint-sampling issue described in Section 4.1. The repository notebook now uses pixel-centre sampling; the C palette patch is documented for a controlled comparison. The original MPI + OpenMP result remains the stronger parallel result on the four-vCPU VM. A complete AMD comparison requires the palette patch, a rootless OpenMPI installation and a rerun of the static and dynamic rows; until then, the cross-environment numbers must be labelled as references.

References

Mandelbrot, B. B. (1982). The Fractal Geometry of Nature. W. H. Freeman.

Message Passing Interface Forum (2021). MPI: A Message-Passing Interface Standard, Version 4.0.

OpenMP Architecture Review Board (2021). OpenMP Application Programming Interface, Version 5.2.

PyTorch Foundation (2026). PyTorch documentation: torch.cuda and ROCm support.

Lab 9 main report: Lab9_Hybrid_MPI_OpenMP_Mandelbrot_Report.docx.

Project repository: https://github.com/YohtHill/mandelbrot-mpi-openmp

Appendix A GPU implementation

The following functions are the core of the AMD port. The first exported run used endpoint sampling; the version below uses pixel-centre sampling, which is the corrected form described in Section 4.1. The complete notebook also contains the environment checks, the C build and the comparison tables.

```
def mandelbrot_torch(width=800, height=600, max_iter=1000,
                     device=DEVICE, dtype=torch.float64):
    xmin, xmax = -2.10, 0.70
    yspan = (xmax - xmin) * height / width
    ymin, ymax = -0.5 * yspan, 0.5 * yspan


    # Sample pixel centres, matching the C code.
    px = torch.arange(width, device=device, dtype=dtype) + 0.5
    py = torch.arange(height, device=device, dtype=dtype) + 0.5
    x = xmin + (xmax - xmin) * px / width
    y = ymax - (ymax - ymin) * py / height
    cr = x.view(1, width).expand(height, width)
    ci = y.view(height, 1).expand(height, width)


    zr = torch.zeros_like(cr)
    zi = torch.zeros_like(ci)
    counts = torch.zeros((height, width), device=device, dtype=torch.int32)
    alive = torch.ones((height, width), device=device, dtype=torch.bool)


    for _ in range(max_iter):
        zr2 = zr * zr
        zi2 = zi * zi
        alive = alive & ((zr2 + zi2) <= 4.0)


        next_zr = zr2 - zi2 + cr
        next_zi = 2.0 * zr * zi + ci


        zr = torch.where(alive, next_zr, zr)
        zi = torch.where(alive, next_zi, zi)
        counts = counts + alive.to(torch.int32)


    return counts
def colourise(counts, max_iter=1000):
    t = torch.sqrt(counts.to(torch.float64) / 200.0).clamp(max=1.0)
    r = (255.0 * torch.clamp(2.0 * t, 0.0, 1.0)).to(torch.uint8)
    g = (255.0 * torch.clamp(2.0 * t - 0.5, 0.0, 1.0)).to(torch.uint8)
    b = (255.0 * torch.clamp(1.0 - 2.0 * t, 0.0, 1.0)).to(torch.uint8)
    rgb = torch.stack([r, g, b], dim=-1).cpu().numpy()
    rgb[counts.cpu().numpy() >= max_iter] = 0
    return rgb


def write_ppm(path, rgb):
    h, w, _ = rgb.shape
    with open(path, "wb") as f:
        f.write(f"P6\n{w} {h}\n255\n".encode("ascii"))
        f.write(rgb.astype("uint8").tobytes())


def fnv1a(data):
    # Same offset basis and prime as the C code (not the standard FNV-1a basis).
    h = 1469598103934665603
    for byte in data:
        h ^= int(byte)
        h = (h * 1099511628211) & 0xFFFFFFFFFFFFFFFF
    return f"{h:016x}"
```

Appendix B Reproduction commands

On a machine with a working OpenMPI installation, the C path is:

```
make
./scripts/run.sh
```

For the AMD container, the notebook is self-contained. The only external dependencies are PyTorch ROCm, numpy and Pillow; gcc is optional, and OpenMPI is needed only for the static and dynamic MPI rows.

If OpenMPI is missing and sudo is blocked, install it through conda-forge:

```
conda install -y -c conda-forge openmpi
# verify
mpicc --version
mpirun --version
mpirun -np 2 hostname
```

Expected toolchain check after a successful install:

```
before: {'gcc': '/usr/bin/gcc', 'mpicc': None, 'mpirun': None}
after : {'gcc': '/usr/bin/gcc', 'mpicc': '/opt/conda/bin/mpicc', 'mpirun': '/opt/conda/bin/mpirun'}
```

Table B1. Dependency checklist for the AMD notebook.

Dependency	Check	Required for
gcc	gcc --version	C serial row
mpicc	mpicc --version	static/dynamic 2x2 rows
mpirun	mpirun --version	static/dynamic 2x2 rows
PyTorch ROCm	torch.version.hip	GPU row
Pillow	import PIL	PNG preview

If the notebook still reports that MPI is missing after installation, restart the Jupyter kernel. The kernel caches the PATH of the process that started it; a new conda prefix is not visible until the kernel restarts. If mpicc is installed but mpirun fails with a root error, set OMPI_ALLOW_RUN_AS_ROOT=1 and OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1 before launching.

Run the notebook from the first cell after installing the toolchain. The comparison table is generated by the final cells and saved to lab9_amd_work/results.json.